Subproblema 6 – Análise léxica da ApiLang

DIM0548 – Engenharia de Linguagens (UFRN/DIMAp) · Referência: APPEL, A. W. *Modern Compiler Implementation in C*, cap. 2.

Produto: o analisador léxico da ApiLang (`apilex`), implementado com **Flex + C** na pasta `lexer/`, e esta documentação. As seções 1 a 5 respondem às questões do enunciado e a seção 6 documenta a implementação.

---

1. O que é um analisador léxico e quais suas principais tarefas?

Definição. O analisador léxico (*scanner*/*lexer*) é a primeira fase do front-end de um compilador. Ele lê o programa como uma sequência de caracteres e o transforma em uma sequência de **tokens**, isto é, as "palavras" da linguagem. Cada token tem uma **classe** (identificador, literal inteiro, palavra reservada `if`, operador `+`…), o **lexema** (o trecho de texto que casou) e, quando for o caso, um **valor semântico** (o número 17, o nome `cont`) e uma **posição** no código-fonte.

Principais tarefas:

1. **Agrupar caracteres em lexemas e classificá-los em tokens**, decidindo onde um termina e o próximo começa: `foreach` é uma palavra reservada, não `for` + `each`; `==` é um operador, não dois `=`.
2. **Descartar o que não interessa à sintaxe**: espaços, tabulações, quebras de linha e comentários.
3. **Converter literais** para a representação interna: `0x1F` vira 31, `"a\tb"` vira a string com uma tabulação real, `'\u{2713}'` vira o código U+2713.
4. **Registrar posições** (linha e coluna), que o parser e as fases seguintes usam nas mensagens de erro.
5. **Detectar e relatar erros léxicos** (caractere inválido, string não terminada, literal malformado) e **se recuperar** para continuar a análise e relatar vários erros de uma vez.
6. **Interagir com a tabela de símbolos**: entregar o nome dos identificadores (ou já inseri-los na tabela) e separar palavras reservadas de identificadores.

Por que separar a análise léxica da sintática? (i) *Simplicidade*: a gramática do parser fica livre de espaços e comentários e trabalha com símbolos terminais prontos. (ii) *Eficiência*: a varredura caractere a caractere é a parte mais cara do front-end, e um autômato finito determinístico é o mecanismo mais rápido para ela. (iii) *Portabilidade*: questões de codificação (ASCII, UTF-8) e de fim de linha ficam isoladas em um único módulo. (iv) Os tokens formam uma **linguagem regular**, para a qual há formalismo (expressões regulares) e geradores automáticos. A estrutura aninhada do programa, que exige uma gramática livre de contexto, fica para o parser.

Na ApiLang o `apilex` faz as seis tarefas: classifica 60 classes de tokens (mais o token `ERROR`); ignora espaços e comentários `//` e `/* */` (com aninhamento); converte inteiros decimais, hexadecimais e binários, com separador `_`, para `int` de 64 bits; converte reais para `double`; traduz os escapes de strings e chars, inclusive `\u{...}` para UTF-8; guarda linha e coluna (contando caracteres UTF-8, não bytes); relata erros no formato `arquivo:linha:coluna: erro léxico: mensagem` sem interromper a análise.

---

2. Como especificar os tokens de uma linguagem de programação?

Os tokens são especificados por **expressões regulares** (ER). Uma ER descreve um conjunto de strings sobre um alfabeto com três operações: concatenação (`ab`), alternativa (`a|b`) e fecho de Kleene (`a*`), além de

abreviações como `[0-9]`, `a+`, `a?` e `.`. Na prática se usam **definições regulares**: ERs com nome, reaproveitadas em outras ERs, como `DIGIT` e `DEC` abaixo.

Como várias ERs podem casar com o mesmo prefixo da entrada, a especificação precisa de **regras de desambiguação**. O Lex/Flex adota as duas descritas por Appel:

- **Casamento mais longo** (*longest match*): vence o maior prefixo que casa com alguma ER. Assim `foreach` é um único token e `<=` não é `<` seguido de `=`.
- **Prioridade da regra**: em caso de empate no tamanho, vence a regra escrita primeiro. Por isso as palavras reservadas vêm antes da regra de identificador: `if` casa com a regra `"if"` e com `{ID}` com o mesmo tamanho, e a primeira vence.

Especificação léxica da ApiLang

Definições regulares (`lexer/apilang.l`):

```
DIGIT    [0-9]
DEC      {DIGIT}(_?{DIGIT})*                    -- 42, 1_000_000
HEX      0[xX][0-9a-fA-F](_?[0-9a-fA-F])*       -- 0x1F
BIN      0[bB][01](_?[01])*                     -- 0b1010_0001
EXP      [eE][+-]?{DEC}
FLOAT    ({DEC}"."{DEC}{EXP}?)|({DEC}{EXP})     -- 3.14, 2.5e-3, 6e2
ID       [A-Za-z_][A-Za-z0-9_]*
ESCAPE   \\[ntr0\\'"]  |  \\u\{[0-9a-fA-F]{1,6}\}
```

Classe	Token(s)	Expressão regular / lexemas
Palavras reservadas	`VAL VAR DEF PUB RETURN IF ELSE FOR FOREACH IN MATCH STRUCT ENUM IMPORT TRUE FALSE`	`val var def pub return if else for foreach in match struct enum import true false`
Tipos primitivos	`T_INT T_FLOAT T_DOUBLE T_DECIMAL T_BOOL T_CHAR T_STR`	`int float double decimal bool char str`
Identificador	`IDENT`	`{ID}`
Inteiro	`INT_LIT`	`{DEC}` | `{HEX}` | `{BIN}` (sem zero à esquerda; até $2^{63}-1$)
Real	`FLOAT_LIT`	`{FLOAT}`
Caractere	`CHAR_LIT`	`'` (um caractere imprimível, UTF-8 ou `{ESCAPE}`) `'`
String	`STRING_LIT`	`"` (qualquer caractere exceto `"`, `\` e quebra de linha, ou `{ESCAPE}`)* `"`
Aritméticos	`PLUS MINUS STAR SLASH PERCENT`	`+ - * / %`
Relacionais	`EQ NE LT LE GT GE`	`== != < <= > >=`
Lógicos	`AND OR NOT`	`&& || !`
Atribuição	`ASSIGN`	`=`
Outros operadores	`ARROW FAT_ARROW RANGE RANGE_INCL SCOPE DOT QUESTION`	`-> => .. ..= :: . ?`
Delimitadores	`LPAREN RPAREN LBRACE RBRACE LBRACKET RBRACKET COMMA SEMICOLON COLON AT`	`( ) { } [ ] , ; : @`

Classe	Token(s)	Expressão regular / lexemas
Ignorados	—	`[ \t\r\n]+`, `"//"[^\n]*`, `/* ... */` (aninháveis)

Uso de cada operador: `->` separa parâmetros e corpo de lambdas (`(u: Usuario) -> u.idade`); `=>` separa padrão e ação nos braços de `match`; `..` / `..=` formam intervalos do `for` (`for i in 0..n`); `::` acessa variantes de `enum` (`ErroApi::NaoEncontrado`); `?` propaga o erro de um `Result`; `@` introduz o decorator de rota dos endpoints (`@get("/usuarios/{id}")`).

Decisões de projeto tomadas na especificação léxica

- **Só os tipos primitivos (e `str`) são palavras reservadas.** `list`, `map`, `set`, `tuple`, `uuid`, `date`, `time`, `datetime`, `Optional`, `Result`, `Some`, `None`, `Ok` e `Err` são **identificadores pré-definidos**, resolvidos na tabela de símbolos. Se `map` e `set` fossem reservados, a chamada de alta ordem `lista.map(f)` (parte da proposta da linguagem) seria um erro de sintaxe, e um campo chamado `date` ou `time` seria proibido. É a mesma escolha de Rust e Kotlin.
- **`!=` é o operador de desigualdade**, e não `<>` (que aparecia na solução 3). O lexer reconhece `<>` e emite um erro com a sugestão `use '!='`, o que vale também para `&` e `|` isolados.
- **Não há literal octal.** Um inteiro com zero à esquerda (`007`) é erro léxico, para evitar a armadilha do C, em que `010 == 8`. Bases alternativas usam prefixo explícito (`0x`, `0b`). A decisão segue a prioridade de confiabilidade e legibilidade.
- **Identificadores são ASCII** (`[A-Za-z_][A-Za-z0-9_]*`). Acentos são aceitos em strings, chars e comentários, mas `preço` como nome é erro léxico com mensagem específica.
- **`null` não é token especial**: lexicamente é um `IDENT`, e o erro aparece na análise semântica ("nome não declarado"), coerente com a linguagem não ter `null`.
- **Comentários de bloco aninham** (`/* a /* b */ c */`), como pede Appel para Tiger. Assim é possível comentar um trecho que já contém comentários.
- **`..` versus números reais**: `0..10` gera `INT_LIT RANGE INT_LIT`, porque `FLOAT` exige dígito depois do ponto. Da mesma forma, `tupla.0` gera `IDENT DOT INT_LIT`.

---

3. Quais abordagens podem ser usadas para implementar um analisador léxico?

1. **Implementação manual (ad hoc).** Um laço com `switch` sobre o caractere atual, funções como `lerNumero()` e `lerIdentificador()`, e um caractere de lookahead. É a escolha de GCC, Clang, Go e Rust, pela velocidade e pelo controle fino de mensagens de erro e de casos irregulares. Exige mais código e é mais fácil errar, porque a especificação fica implícita no código.
2. **Autômato finito codificado à mão.** Constrói-se o AFD dos tokens e ele é implementado como uma **tabela de transições** `T[estado][caractere]` mais um vetor de estados finais, ou como código com um `case` por estado. É o modelo da seção 2.3 de Appel.

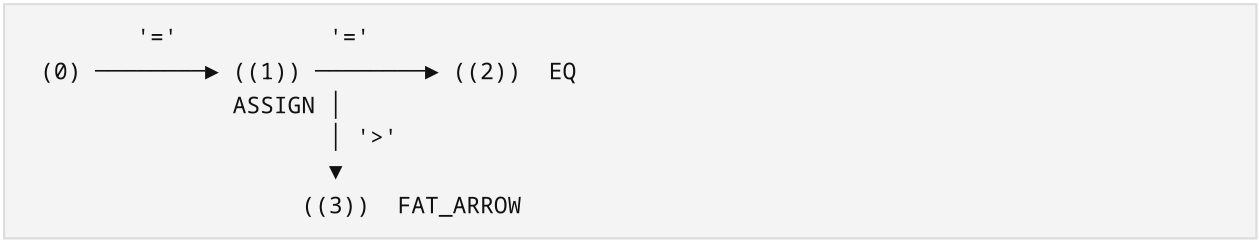
3. **Gerador de analisadores léxicos.** A partir das ERs e das ações associadas, uma ferramenta gera o código do autômato: **Lex/Flex** (C), **JFlex** (Java), **ML-Lex/ocamllex**, **re2c**, ou o lexer integrado do **ANTLR**. Internamente o gerador segue o caminho clássico:

    - ER → **AFN** (construção de Thompson);
    - AFN → **AFD** (construção de subconjuntos, com ε-fecho);
    - **minimização** do AFD (fusão de estados equivalentes);

- compressão das tabelas.

Na execução, o autômato roda até não haver mais transição, **lembrando o último estado final** visto. Ao travar, ele retrocede até esse ponto e executa a ação da regra correspondente. É assim que o *longest match* é implementado.

Exemplo do AFD gerado para `=`, `==` e `=>`:



Exemplo de retrocesso para os inteiros/reais em `0..10`:



Lendo `0`, o autômato vai para o estado final 1. Lendo `.`, vai para o estado 2, que não é final. O segundo `.` não tem transição a partir de 2, então o autômato retrocede ao último estado final, emite `INT_LIT 0` e recomeça em `..`.

Escolha da ApiLang: gerador Flex. (i) A especificação fica **declarativa e legível**: a tabela da seção 2 corresponde quase linha a linha a `apilang.l`, o que facilita manutenção e avaliação. (ii) O Flex garante *longest match* e prioridade de regra de forma correta e gera um AFD eficiente. (iii) Integra-se diretamente ao **Bison**, que será usado no analisador sintático: o parser chama `yylex()` e lê `yylval` e `yylloc`, e os códigos dos tokens já começam em 258, como o Bison espera. (iv) É a ferramenta usada no livro de referência. O custo é ter menos controle sobre casos irregulares, resolvido com **condições de início** (`%x STR COMMENT`) para strings e comentários aninhados e com **regras de erro** explícitas.

---

4. O que fazer com a lista de tokens gerados a partir de um programa?

A lista de tokens é a **entrada do analisador sintático**. Em um compilador real ela normalmente nem é materializada: o parser chama o lexer **sob demanda**, um token por vez (`yyparse()` chama `yylex()` sempre que precisa do próximo símbolo de lookahead). Por isso o lexer não precisa guardar o programa inteiro na memória.

Com cada token o parser:

- usa a **classe** como símbolo terminal da gramática para decidir a próxima ação (derivar, no caso descendente; empilhar ou reduzir, no caso ascendente) e assim **verificar se o programa é sintaticamente válido**;
- usa o **valor semântico** (`yylval`) para montar a árvore: o nome de um `IDENT` vira uma folha que depois será ligada à tabela de símbolos, e o valor de um `INT_LIT` vira uma constante;
- usa a **posição** (`yylloc`) para relatar erros sintáticos e anotar a árvore, para que as fases seguintes (verificação de tipos, imutabilidade, exaustividade de `match`) também apontem linha e coluna.

O token `ERROR` emitido pelo `apilex` permite ao parser tratar o erro léxico como um erro sintático recuperável (produção `error` do Bison), em vez de abortar.

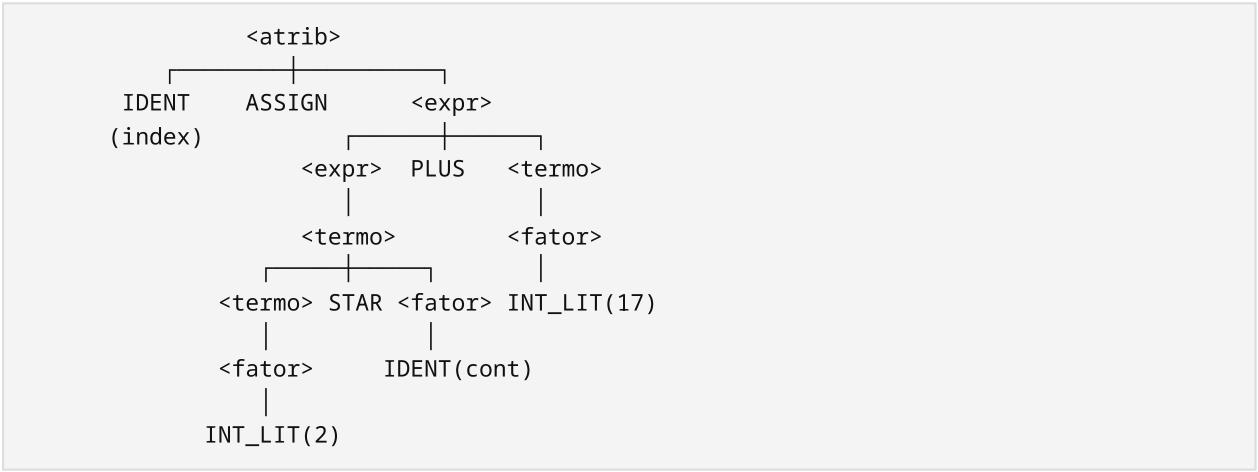
Neste subproblema, como o parser ainda não existe, o driver `apilex` **imprime** a lista para validação:

```
$ ./apilex testes/01_enunciado_p5.api        # index = 2 * cont + 17
LIN:COL   TOKEN        LEXEMA / VALOR
1:1       IDENT        index
1:7       ASSIGN       =
1:9       INT_LIT      2            = 2
1:11      STAR         *
1:13      IDENT        cont
1:18      PLUS         +
1:20      INT_LIT      17           = 17
-- 7 tokens, 0 erro(s) léxico(s)
```

---

5. Árvore definida pelas regras da gramática: utilidade e quando é construída

Para cada programa válido existe uma **árvore de derivação** (*parse tree*). A raiz é o símbolo inicial, os nós internos são não terminais (cada nó com seus filhos corresponde à aplicação de uma regra) e as folhas, lidas da esquerda para a direita, são exatamente a sequência de tokens produzida pelo lexer. Por exemplo, para `index = 2 * cont + 17`, com uma gramática de expressões estratificada por precedência:



Utilidade:

- **Explicita a estrutura do programa**, que não aparece na sequência linear de tokens: mostra que `2 * cont` é agrupado antes do `+`, ou seja, a **precedência e a associatividade** codificadas na gramática. Uma gramática ambígua admite duas árvores para o mesmo programa, e portanto dois significados.
- **Serve de base para todas as fases seguintes.** Normalmente ela é condensada em uma **árvore sintática abstrata (AST)**, que descarta nós de cadeia (`<termo> → <fator>`) e tokens que só têm papel sintático (`;`, `{`, `}`). Sobre a AST se faz a **análise semântica** (resolução de escopo, tipos, imutabilidade de `val`, exaustividade de `match` na ApiLang), por exemplo decorando os nós com atributos como na gramática de atributos do Subproblema 5. Também se fazem as **otimizações** e a **geração de código** intermediário e nativo.

Quando é construída: durante a **análise sintática**, à medida que o parser consome os tokens.

- Em um parser **descendente** (LL, recursivo), a árvore cresce da raiz para as folhas, em pré-ordem; cada chamada de função corresponde a um nó.
- Em um parser **ascendente** (LR/LALR, como o gerado pelo Bison), ela cresce das folhas para a raiz: cada **redução** cria um nó cujos filhos são os símbolos do lado direito da regra (é uma derivação mais à direita ao contrário).

Na prática a árvore de derivação completa costuma ficar **implícita**: ela corresponde à sequência de chamadas ou reduções do parser, e o que é de fato montado em memória, por meio das **ações semânticas** associadas às regras, é a AST. Esse é o plano para a ApiLang: no próximo subproblema, as ações do Bison construirão a AST a partir dos tokens entregues pelo `apilex`.

---

6. Documentação da implementação (`lexer/`)

Arquivos

Arquivo	Conteúdo
`apilang.l`	Especificação Flex: definições regulares, regras e ações, rotinas auxiliares (posição, buffer de strings, conversão de literais, erros)
`tokens.h`	Enumeração das classes de token (a partir de 258, compatível com Bison), `yylval` (valor semântico) e `yylloc` (posição)
`main.c`	Driver: abre o arquivo, chama `yylex()` até o EOF e imprime a lista de tokens
`Makefile`	`make` compila, `make test` roda os testes, `make clean` limpa
`exemplos/`	`quicksort.api` (programa do Subproblema 5), `api_usuarios.api` (endpoints, `struct`, `enum`, `match`, lambdas) e `erros_lexicos.api`
`testes/`	Casos de teste `.api` com a saída esperada (`.esperado`) e o script `rodar_testes.sh`

Compilação e uso

```
$ cd lexer
$ make                          # flex apilang.l && gcc ... -o apilex
$ ./apilex exemplos/quicksort.api
$ ./apilex < programa.api       # também lê da entrada padrão
$ make test                     # 9/9 testes passaram
```

Requisitos: `flex` 2.6 e `gcc` (C11). Código de saída: `0` sem erros, `1` se houve erro léxico, `2` se o arquivo não pôde ser aberto.

Funcionamento interno

- **Posição:** a macro `YY_USER_ACTION` chama `update_loc()` antes de cada ação. Ela grava em `yylloc` a linha e a coluna de início e de fim do lexema e atualiza a posição corrente. Bytes de continuação UTF-8 não contam coluna.
- **Strings:** ao ver `"`, o lexer entra na condição exclusiva `STR`. Cada trecho ou escape é acrescentado a um buffer dinâmico já traduzido (`\n` vira o byte 10, `\u{1F600}` vira os 4 bytes UTF-8). Ao fechar a aspa, retorna `STRING_LIT` com a posição da aspa de abertura.

- **Comentários de bloco:** a condição `COMMENT` mantém um contador de profundidade, incrementado em `/*` e decrementado em `*/`. Volta ao estado inicial quando o contador chega a 0.
- **Literais numéricos:** os `_` são removidos e o valor é convertido com `strtoull`/`strtod`, verificando o limite de `int` (64 bits com sinal) e de `double`.
- **Palavras reservadas:** cada uma tem uma regra própria antes de `{ID}`. Pela prioridade de regra, `if` é `IF`; pelo casamento mais longo, `iff` e `if_` são `IDENT`.

Erros léxicos tratados

Os erros são relatados em `stderr` no formato `arquivo:linha:coluna: erro léxico: mensagem`. A **recuperação** é em modo pânico local: o lexema inválido é descartado, emite-se o token `ERROR` e a análise continua no próximo caractere. Assim, um único passe relata todos os erros, como mostra `exemplos/erros_lexicos.api` (14 erros, um por linha).

Situação	Exemplo	Mensagem
Caractere fora do alfabeto	`#`, `$`, `` ` ``, byte inválido	`caractere inválido '#'`
Não-ASCII fora de string/comentário	`val preço`	`'preço': caracteres não-ASCII só são permitidos em strings, chars e comentários`
Operador inexistente	`a <> b`, `a & b`	`operador '<>' não existe na ApiLang; use '!='`
Inteiro com zero à esquerda	`007`	`literal inteiro '007' com zero à esquerda (não há octal…)`
Número malformado	`12abc`, `1e`, `0xZZ`, `3.14xyz`	`literal numérico malformado '12abc'`
Inteiro fora da faixa	`99999999999999999999`	`literal inteiro '…' excede o tipo int (64 bits)`
Char vazio, com vários caracteres ou não terminado	`''`, `'ab'`, `'a`	`literal de caractere vazio` etc.
Escape inválido	`"\q"`, `'\u{D800}'`	`sequência de escape inválida '\q' em string`
String não terminada	`"abc` + quebra de linha ou EOF	`string não terminada (…)`, apontando a aspa de abertura
Comentário não terminado	`/* ...` até o EOF	`comentário de bloco não terminado (aberto aqui)`

Exemplo de saída:

```
$ ./apilex exemplos/erros_lexicos.api
exemplos/erros_lexicos.api:2:14: erro léxico: literal inteiro '007' com zero à esquerda
(não há octal; use 0x/0b ou remova o zero)
exemplos/erros_lexicos.api:3:14: erro léxico: literal numérico malformado '12abc'
exemplos/erros_lexicos.api:8:17: erro léxico: operador '<>' não existe na ApiLang; use
'!='
exemplos/erros_lexicos.api:10:5: erro léxico: 'preço': caracteres não-ASCII só são
permitidos em strings, chars e comentários (identificadores usam [A-Za-z0-9_])
exemplos/erros_lexicos.api:13:14: erro léxico: string não terminada (quebra de linha
antes de fechar aspas)
...
-- 103 tokens, 14 erro(s) léxico(s)
```

Testes

`testes/rodar_testes.sh` executa o `apilex` em cada `testes/*.api` e compara a saída completa (tokens e erros) com o `.esperado` correspondente. Os casos cobrem:

- o exemplo `index = 2 * cont + 17` do Subproblema 5;
- casamento mais longo e prioridade (`for`/`foreach`/`forx`, `=`/`==`/`=>`, `.`/`..`/`..=`);
- todas as formas de literal numérico, incluindo `0..10` e `tupla.0`;
- strings e chars com escapes e UTF-8;
- comentários de linha, de bloco e aninhados;
- erros com recuperação;
- os três programas de `exemplos/`.

Resultado atual: **9/9 testes passaram**.